

Cahier des Charges

Outil Speech-to-Text

Projet Qwen3-ASR

Date : 10/09/2026

Version : 1.0

Statut : En cours de developpement



Table des Matieres

1 Presentation du Projet

- 1.1 Contexte et Objectifs
- 1.2 Perimetre Fonctionnel
- 1.3 Technologies Utilisees

2 Architecture Technique

- 2.1 Architecture en Couches
- 2.2 Flux de Donnees
- 2.3 Stack Technologique Detaillee
- 2.4 Organisation du Code

3 Modele Conceptuel de Donnees (MCD)

- 3.1 Entite Principale : TranscriptionJob
- 3.2 Diagramme Entite-Association
- 3.3 Relations et Dependances
- 3.4 Cycle de Vie d'un Job
- 3.5 Regles de Gestion

4 Fonctionnalites Detaillees

- 4.1 Module Transcription
- 4.2 Module Correction
- 4.3 Module Statistiques
- 4.4 Module Export

5 Moteur ASR Qwen3-ASR

- 5.1 Caracteristiques du Modele
- 5.2 Service Qwen3Service
- 5.3 Worker Asynchrone

6 Interface Utilisateur

- 6.1 Dashboard
- 6.2 Page Upload
- 6.3 Page Detail
- 6.4 Page Correction
- 6.5 Page Statistiques

7 Gestion de Projet

- 7.1 Methodologie
- 7.2 Organisation de l'Equipe
- 7.3 Planification et Jalons
- 7.4 Gestion des Risques

8 Exigences Non Fonctionnelles

- 8.1 Performance
- 8.2 Securite
- 8.3 Compatibilite
- 8.4 Maintenance

A Annexe A : Configuration du Projet

B Annexe B : Routes URL

C Annexe C : Dependances

D Annexe D : Enums

E Annexe E : Endpoints API

F Annexe F : Deploiement

G Annexe G : Gestion des Erreurs

1. Presentation du Projet

1.1 Contexte et Objectifs

Ce projet s'inscrit dans le cadre d'un stage portant sur le developpement d'un outil de Speech-to-Text (transcription automatique de la parole). L'objectif est de concevoir, developper et deployer une application web complete permettant aux utilisateurs de transcrire des fichiers audio de maniere automatique grace au modele Qwen3-ASR, un modele de reconnaissance automatique de la parole (ASR) developpe par Alibaba/Qwen.

Le projet vise a fournir une solution accessible, performante et conviviale pour la transcription audio, avec des fonctionnalites avancees comme la detection automatique de langue, la generation de timestamps (sous-titres SRT), et un systeme de correction manuelle cote a cote.

Objectifs Principaux

Fournir un outil web de transcription audio precis et rapide, avec interface de correction, export multi-format, et statistiques de performance (CER/WER).

1.2 Perimetre Fonctionnel

Le perimetre fonctionnel du projet couvre les domaines suivants :

- Transcription automatique de fichiers audio (WAV, MP3, FLAC, OGG, M4A)
- Detection automatique ou manuelle de la langue de l'audio
- Generation de timestamps precis pour chaque segment
- Correction manuelle du texte transcrit avec vue cote a cote
- Statistiques de performance (CER, WER, temps de traitement)
- Export dans multiple formats (TXT, SRT, JSON)
- Gestion des jobs de transcription (file d'attente, suivi, historique)
- Interface web moderne et responsive avec theme sombre

1.3 Technologies Utilisees

Categorie	Technologie	Version/Details
Backend	Django	5.x - Framework Python
Modele ASR	Qwen3-ASR	1.7B parametres - Alibaba/Qwen
Frontend	Bootstrap 5	CSS framework responsive
Graphiques	Chart.js	Visualisation des metriques
Base de donnees	SQLite	Developpement / PostgreSQL production
Python	3.11+	Langage principal
Async	Worker synchrone	Traitementsequentiel des jobs
Alignement	ForcedAligner	Timestamps precis par segment

2. Architecture Technique

2.1 Architecture en Couches

L'application suit une architecture en couches (layers) typique de Django, separent clairement la presentation, la logique metier et l'accès aux donnees.

```
+-----+
|          COUCHE PRESENTATION          |
|  Templates HTML + Bootstrap 5 + Chart.js  |
|  JavaScript (fetch API, polling, diff)    |
+-----+
|          COUCHE LOGIQUE (Views)        |
|  views.py : Upload, Detail, Correction, Stats  |
|  urls.py : Routage HTTP                  |
+-----+
|          COUCHE SERVICE ASR             |
|  qwen_service.py : Singleton Qwen3Service    |
|  run_worker.py : Worker de traitement async  |
+-----+
|          COUCHE DONNEES                 |
|  models.py : TranscriptionJob (Django ORM)   |
|  SQLite / PostgreSQL                     |
+-----+
```

2.2 Flux de Donnees

Le diagramme suivant illustre le flux de donnees complet depuis l'upload du fichier audio jusqu'a la visualisation des resultats :



2.3 Stack Technologique Detaillee

Backend Django :

- Framework : Django 5.x avec ORM puissant
- Routing : urls.py avec namespace 'transcriptions'
- Templates : Moteur de template Django + {% extends %} + {% block %}
- Static : Fichiers CSS/JS servis par Django en dev, Nginx en prod
- Media : Fichiers audio stockes dans media/uploads/

Modele ASR Qwen3-ASR :

- Developpe par Alibaba/Qwen (equipe Qwen Team)

- 1.7 milliard de parametres, entraine sur 200 000+ heures audio
- Supporte 11 langues : EN, ZH, DE, FR, ES, IT, JA, RU, PT, KO, AR
- ForcedAligner integre pour les timestamps precis
- Precisions : bfloat16 (GPU) ou float32 (CPU)

2.4 Organisation du Code

```
Qwen3-ASR/
+-- pyproject.toml          # Configuration projet + dependances
+-- README.md              # Documentation du modele
+-- generate_cahier_des_charges.py # Ce script
+-- .venv/                 # Environnement virtuel Python
+-- webapp/                # Application Django
    +-- manage.py          # CLI Django
    +-- qwenweb/           # Configuration Django
        | +-- settings.py  # Parametres principaux
        | +-- urls.py      # Routes principales
        | +-- wsgi.py      # Point d'entree WSGI
    +-- transcriptions/    # Application principale
        +-- models.py      # Modele de donnees (TranscriptionJob)
        +-- views.py       # Vues Django (Pages + API)
        +-- urls.py        # Routes de l'application
        +-- qwen_service.py # Service singleton Qwen3-ASR
        +-- metrics.py     # Calcul CER/WER
        +-- templates/     # Templates HTML
            | +-- base.html
            | +-- dashboard.html
            | +-- upload.html
            | +-- detail.html
            | +-- correction.html
            | +-- stats.html
        +-- static/        # Fichiers statiques
            | +-- css/qwen.css # Styles personnalisés
        +-- management/    # Commandes personnalisées
            +-- commands/run_worker.py
```

3. Modele Conceptuel de Donnees (MCD)

3.1 Entite Principale : TranscriptionJob

Le modele de donnees est centree sur l'entite TranscriptionJob qui encapsule toutes les informations relatives a une transcription audio.

Champ	Type	Description
id	UUID (PK)	Identifiant unique automatique
created_at	DateTime	Date et heure de creation
updated_at	DateTime	Derniere mise a jour
status	Char(20)	pending / running / completed / failed
language	Char(10)	Code langue (auto/fr/en/...)
audio_file	FileField	Chemin vers le fichier audio
original_filename	Char(255)	Nom original du fichier
transcript_text	Text	Texte brut de la transcription
corrected_text	Text	Texte corrige par l'utilisateur
segments	JSONField	Segments avec timestamps [{start,end,text}]
timestamps	Boolean	Generer les timestamps (default: True)
max_new_tokens	Integer	Nombre max de tokens (64-2048)
processing_time	Float	Duree de traitement en secondes
completed_at	DateTime	Date de fin de traitement
model_used	Char(100)	Nom du modele utilise
cer_score	Float	Character Error Rate (0-1)
wer_score	Float	Word Error Rate (0-1)
word_count	Integer	Nombre de mots
char_count	Integer	Nombre de caracteres
duration	Float	Duree de l'audio en secondes
error_message	Text	Message d'erreur si echec

3.2 Diagramme Entite-Association



3.5 Regles de Gestion

Regles de gestion detaillees liees au modele de donnees :

- RG01 : Chaque transcription est identifiee par un UUID unique, non modifiable
- RG02 : Le statut passe de pending a running lors du demarrage du traitement
- RG03 : Un job ne peut passer de completed a failed que si une erreur survient
- RG04 : Le fichier audio est stocke dans media/uploads/ avec le nom UUID
- RG05 : Le transcript_text est rempli uniquement apres succes du traitement
- RG06 : Le corrected_text est optionnel, uniquement apres intervention humaine
- RG07 : Les segments JSON contiennent {start, end, text} pour chaque phrase
- RG08 : Le timestamps active la generation des timestamps au moment de la transcription
- RG09 : Le max_new_tokens est compris entre 64 et 2048, default 512
- RG10 : Le processing_time est calcule comme completed_at - created_at
- RG11 : Le cer_score et wer_score sont optionnels, calcules uniquement si corrected_text existe
- RG12 : La suppression d'un job supprime egalement le fichier audio associe

4. Fonctionnalites Detaillees

4.1 Module Transcription

Le module de transcription gere le flux complet depuis l'upload jusqu'au resultat :

1. Upload : l'utilisateur selectionne un fichier audio (WAV, MP3, FLAC, OGG, M4A)
2. Validation : verification du type et de la taille du fichier (max 2 Go)
3. Creation du job : un TranscriptionJob est cree avec le statut 'pending'
4. Envoi au worker : le job est traite de maniere sequentielle
5. Traitement ASR : Qwen3Service transcrit l'audio en texte
6. Alignement : le ForcedAligner genere les timestamps precis
7. Stockage : le transcript_text et les segments sont sauvegardes
8. Resultat : l'utilisateur peut consulter le texte et les segments

4.2 Module Correction

Le module de correction permet d'ameliore la qualite de la transcription :

- Vue cote a cote : texte original (gauche) vs texte corrige (droite)
- Editeur de texte : zone de texte pour modifier la transcription
- Sauvegarde : le corrected_text est stocke dans le meme job
- Calcul des metriques : CER/WER automatiques apres correction
- Historique : garder la trace des modifications

4.3 Module Statistiques

Le module de statistiques fournit des metriques de performance :

- CER (Character Error Rate) : taux d'erreur par caractere
- WER (Word Error Rate) : taux d'erreur par mot
- Temps de traitement : duree totale de la transcription
- Statistiques par fichier : nombre de mots, caracteres, duree
- Graphiques : visualisation avec Chart.js (barres, courbes)

4.4 Module Export

Le module d'export permet de telecharger les resultats en differents formats :

Format	Contenu	Usage
TXT	Texte brut de la transcription	Lecture simple, partage
SRT	Texte avec timestamps (sous-titres)	Montage video, sous-titres
JSON	Structure complete (segments, metriques)	Integration API, analyse

5. Moteur ASR Qwen3-ASR

5.1 Caracteristiques du Modele

Le modele Qwen3-ASR est un modele de reconnaissance automatique de la parole :

- Developpe par Alibaba/Qwen (equipe Qwen Team)
- Architecture : Transformer encoder-decoder optimise pour l'ASR
- Taille : 1.7 milliard de parametres
- Donnees d'entrainement : 200 000+ heures audio multilingues
- Langues supportees : EN, ZH, DE, FR, ES, IT, JA, RU, PT, KO, AR
- Precision : bfloat16 (GPU) ou float32 (CPU)
- Features : timestamps, diarisation, detection de langue
- ForcedAligner : alignement force pour des timestamps precis

5.2 Service Qwen3Service (Singleton)

Le service Qwen3Service est implemente comme un Singleton pour optimiser l'utilisation de la memoire GPU/CPU :

- Initialisation paresseuse : le modele est charge uniquement a la premiere utilisation
- Cache en memoire : une seule instance du modele est garde en vie
- Device automatique : detection CUDA/CPU au demarrage
- Parametres configurables : language, timestamps, max_new_tokens
- Gestion d'erreurs : fallback CPU si GPU indisponible
- Thread-safe : utilisation de locks pour les appels concurrents

5.3 Worker Asynchrone (run_worker.py)

Le worker traite les jobs de transcription de maniere sequentielle :

1. Demarrage : python manage.py run_worker
2. Boucle infinie : le worker surveille les jobs en attente (pending)
3. Selection : le job le plus ancien (pending) est selectionne
4. Traitement : le statut passe a running, puis le modele transcrit
5. Sauvegarde : transcript_text, segments, metrics sont stockes
6. Completion : le statut passe a completed avec le processing_time
7. Erreur : le statut passe a failed avec le message d'erreur
8. Mode --once : executer une seule iteration puis quitter

6. Interface Utilisateur

6.1 Dashboard (Accueil)

Le dashboard est la page d'accueil affichant un aperçu du système :

- Statistiques globales : nombre de transcriptions, taux de succès
- Derniers jobs : liste des 5 derniers jobs avec statut
- Actions rapides : bouton pour démarrer une nouvelle transcription
- Graphiques : évolution des transcriptions dans le temps

6.2 Page Upload

La page d'upload permet de soumettre un fichier audio pour transcription :

- Zone de dépôt : drag & drop ou clic pour sélectionner
- Types acceptés : WAV, MP3, FLAC, OGG, M4A (max 2 Go)
- Options : langue (auto/fr/en/...), timestamps (on/off), max tokens
- Soumission : création du job et redirection vers la page détail

6.3 Page Detail

La page détail affiche les informations complètes d'un job :

- Statut : badge coloré (pending/running/completed/failed)
- Métadonnées : langue, durée, timestamps, tokens, date
- Texte brut : affichage du transcript_text
- Segments : liste des segments avec timestamps
- Actions : corriger, exporter (TXT/SRT/JSON), supprimer

6.4 Page Correction

La page correction permet d'améliorer la transcription :

- Vue côte à côte : texte original (gauche) vs éditeur (droite)
- Éditeur : zone de texte avec mise en forme
- Sauvegarde : enregistrement du corrected_text
- Métriques : calcul automatique du CER/WER

6.5 Page Statistiques

La page statistiques fournit des métriques détaillées :

- CER/WER global : taux d'erreur moyen
- Temps moyen : durée moyenne de traitement
- Statistiques par fichier : nombre de mots, caractères, durée
- Graphiques : barres (CER/WER), courbes (évolution)

7. Gestion de Projet

7.1 Methodologie

Le projet suit une methodologie agile avec des itérations de 1-2 semaines :

- Sprint 1 : Mise en place de l'environnement, integration du modele ASR
- Sprint 2 : Developpement de l'upload et du worker de traitement
- Sprint 3 : Interface de detail et correction
- Sprint 4 : Statistiques, export, et tests
- Sprint 5 : Optimisation, documentation, et deploiement

7.2 Organisation de l'Equipe

Le projet est developpe dans le cadre d'un stage :

Role	Responsabilites	Outils
Developpeur (Stagiaire)	Developpement, tests, documentation	VS Code, Git, Django
Tuteur de stage	Encadrement technique, validation	Reunions, revues de code
Utilisateurs finaux	Tests fonctionnels, feedback	Navigateur web

Outils de Gestion de Projet

- Git + GitHub : version control, branches feature/fix, pull requests
- VS Code : IDE principal avec extensions Python/Django
- Django manage.py : CLI pour migrations, tests, serveur dev
- GitHub Issues : suivi des bugs et des fonctionnalites
- README.md : documentation technique du projet
- Cahier des charges (ce document) : reference fonctionnelle et technique

Processus de Developpement

1. Analyse des besoins : lecture du modele Qwen3-ASR, definition du perimetre
2. Conception : MCD, architecture, interfaces (wireframes textuels)
3. Developpement iteratif : sprints de 1-2 semaines par module
4. Tests : unitaires (Django TestCase), integration, fonctionnels
5. Revue de code : verification de la qualite avant merge
6. Deploiement : serveur Django dev (manage.py runserver)
7. Documentation : README, docstrings, cahier des charges
8. Maintenance : corrections de bugs, optimisations, nouvelles fonctionnalites

7.3 Planification et Jalons

Phase	Duree	Livrables	Criteres de Validation
Phase 1 : Setup	1 sem	Env. dev, modele charge	Serveur demarre, transcription basique OK
Phase 2 : Core	2 sem	Upload, worker, detail	Upload -> transcription -> affichage OK
Phase 3 : UI	2 sem	Correction, stats, export	Toutes les pages fonctionnelles
Phase 4 : Polish	1 sem	Tests, doc, optimisation	13+ tests passent, README complet

7.4 Gestion des Risques

Identification et mitigation des risques potentiels :

Risque	Probabilite	Impact	Mitigation
Memoire GPU insuffisante	Moyenne	Eleve	Fallback CPU, reduction batch size
Performance lente (CPU)	Haute	Moyen	Optimisation, timeout, mode async
Formats audio non supportes	Faible	Moyen	Validation cote serveur, message clair
Erreurs modele ASR	Moyenne	Eleve	Gestion d'erreurs, retry, fallback
Perte de donnees	Faible	Eleve	Backup BDD, sauvegarde fichiers media
Securite (fichiers)	Faible	Eleve	Validation type/taille, stockage securise
Deploiement complexe	Moyenne	Moyen	Docker, scripts d'installation, doc

8. Exigences Non Fonctionnelles

8.1 Performance

- Temps de reponse upload : < 5 secondes pour un fichier de 10 Mo
- Temps de transcription : depend de la duree de l'audio et du device
- Charge simultanee : traitement sequentiel (1 job a la fois)
- Optimisation memoire : modele charge une seule fois en memoire

8.2 Securite

- Validation des fichiers : type MIME et taille maximum (2 Go)
- Stockage securise : fichiers dans media/uploads/ avec noms UUID
- CSRF : protection Django contre les attaques Cross-Site Request Forgery
- Pas d'authentification : application interne (a ajouter en production)

8.3 Compatibilite

- Navigateurs : Chrome, Firefox, Safari, Edge (dernieres versions)
- Responsive : adaptation aux tablettes et mobiles
- OS : Windows, Linux, macOS (cote serveur)
- Python : 3.11+ (recommande 3.12+)

8.4 Maintenance

- Documentation : README.md complet, docstrings dans le code
- Tests : 13 tests unitaires couvrant les fonctionnalites principales
- Versioning : Git avec branches feature/fix, tags de version
- Deploiement : scripts d'installation, Docker (a prévoir)
- Monitoring : logs Django, metriques de performance

A. Annexes

Annexe A : Configuration du Projet

Parametres principaux de Django (settings.py) :

Parametre	Valeur	Description
BASE_DIR	Path(__file__).resolve().parent.parent	Racine du projet Django
SECRET_KEY	Cle secrete Django	Securite des sessions CSRF
DEBUG	True (dev)	Mode debug avec pages d'erreur detaillees
ALLOWED_HOSTS	["127.0.0.1", "localhost"]	Hosts autorises
INSTALLED_APPS	[..., "transcriptions"]	Application principale
MEDIA_ROOT	BASE_DIR / "media"	Dossier de stockage audio
MEDIA_URL	/media/	URL de service des fichiers media
DATABASES	SQLite	Base de donnees locale
STATIC_URL	/static/	Fichiers statiques (CSS/JS)
DEFAULT_AUTO_FIELD	django.db.models.BigAutoField	Type de cle primaire

Configuration du Service ASR

- Modele par default : Qwen3-ASR-1.7B (changeable dans qwen_service.py)
- Device automatique : cuda:0 si disponible, sinon cpu
- Dtype : bfloat16 (reduit la consommation memoire)
- ForcedAligner : active si timestamps=True
- Max tokens : configurable par l'utilisateur (64-2048, default 512)
- Batch size : 1 (un seul fichier a la fois)

Annexe B : Routes URL

Route	Vue	Description
/	dashboard	Page d'accueil avec statistiques
/upload/	upload_file	Page d'upload de fichier audio
/jobs/	jobs_list	Liste de tous les jobs
/jobs/<uuid>/	job_detail	Detail d'un job specifique
/corrections/<uuid>/	correction_view	Page de correction
/stats/	stats_view	Statistiques globales
/stats/metrics/<uuid>/	file_metrics	Metriques par fichier
/api/job/create/	api_create_job	API : creer un job (POST)
/api/status/<uuid>/	api_job_status	API : status d'un job (GET)
/api/corrections/<uuid>/	api_correction	API : sauvegarder correction
/api/export/<uuid>/<fmt>/	api_export	API : exporter en TXT/SRT/JSON

Annexe C : Dependances

Dependances principales (pyproject.toml) :

Package	Version	Usage
django	>=5.0	Framework web
qwen3-asr	>=1.0	Modele ASR Qwen3
qwen-audio	>=1.0	Alignement force (ForcedAligner)
torch	>=2.0	PyTorch (deep learning)
torchaudio	>=2.0	Traitement audio PyTorch
transformers	>=4.0	Hugging Face Transformers
fpdf2	>=2.0	Generation de PDF

Annexe D : Enums et Valeurs

Enumerations utilisees dans le projet :

- Status : pending, running, completed, failed
- Language : auto, fr, en, de, es, it, ja, ru, pt, ko, ar
- Export formats : txt, srt, json
- Device : cpu, cuda:0, cuda:1, ...

Annexe E : Endpoints API

Documentation des endpoints API REST :

```
POST /api/job/create/
Body (multipart/form-data) :
- audio_file : fichier audio (requis)
- language : code langue (optionnel, default: auto)
- timestamps : true/false (optionnel, default: true)
- max_new_tokens : 64-2048 (optionnel, default: 512)
Response 201 :
{
  "job_id": "uuid",
  "status": "pending",
  "url": "/jobs/<uuid>/"
}

GET /api/status/<uuid>/
Response 200 :
{
  "job_id": "uuid",
  "status": "completed",
  "transcript_text": "...",
  "segments": [...],
  "processing_time": 12.5,
  "cer_score": 0.05
}

POST /api/corrections/<uuid>/
Body JSON :
{
  "corrected_text": "texte corrige"
}
Response 200 :
{
  "status": "corrected",
  "cer_score": 0.02
}

GET /api/export/<uuid>/txt/
Response 200 : fichier TXT en telechargement

GET /api/export/<uuid>/srt/
Response 200 : fichier SRT en telechargement

GET /api/export/<uuid>/json/
Response 200 : fichier JSON en telechargement
```

Annexe F : Deploiement et Demarrage

Etapes pour installer et demarrer l'application :

Installation

1. Cloner le depot : `git clone <url> Qwen3-ASR`
2. Creer un environnement virtuel : `python -m venv .venv`
3. Activer le venv : `.venv/Scripts/activate` (Windows)
4. Installer les dependances : `pip install -e .` (via `pyproject.toml`)
5. Installer le modele : `python -c "from qwen_asr import Qwen3ASRModel; Qwen3ASRModel.from_pretrained()"`
6. Appliquer les migrations : `python manage.py migrate`
7. Creer un superuser (optionnel) : `python manage.py createsuperuser`

Demarrage

1. Terminal 1 : Serveur Django - `python manage.py runserver 127.0.0.1:8001 --noreload`
2. Terminal 2 : Worker ASR - `python manage.py run_worker`
3. Ouvrir le navigateur : `http://127.0.0.1:8001/`
4. Verifier le status : `http://127.0.0.1:8001/ (dashboard)`

Commandes Utiles

Commande	Description
<code>python manage.py runserver</code>	Demarrer le serveur de developpement
<code>python manage.py run_worker</code>	Demarrer le worker ASR (boucle continue)
<code>python manage.py run_worker --once</code>	Executer une seule iteration du worker
<code>python manage.py migrate</code>	Appliquer les migrations de la BDD
<code>python manage.py createsuperuser</code>	Creer un administrateur Django
<code>python manage.py shell</code>	Console Python Django
<code>python manage.py test</code>	Lancer les tests unitaires
<code>pip install -e .</code>	Installer le projet en mode editable

Deploiement Production

Pour un deploiement en production, les elements suivants sont recommandes :

- Serveur : Gunicorn ou uWSGI (pas le serveur de developpement Django)
- Reverse proxy : Nginx ou Apache pour les fichiers statiques et media
- Base de donnees : PostgreSQL ou MySQL (remplacer SQLite)
- Cache : Redis pour le cache et les sessions
- File d'attente : Celery + Redis pour le worker asynchrone
- HTTPS : Let's Encrypt ou certificat commercial
- Monitoring : logs Django, metriques de performance
- Sauvegardes : backup regulier de la BDD et des fichiers media

Annexe G : Gestion des Erreurs

Le systeme gere les erreurs suivantes :

Erreur	Cause	Traitement
Fichier non supporte	Extension non reconnue	Validation cote serveur, message d'erreur
Fichier trop volumineux	> 2 Go	Limitation avant upload, message d'erreur
Modele non charge	Memoire insuffisante	Fallback CPU, erreur 500
Transcription echoue	Audio corrompu ou trop long	Statut failed + message d'erreur
Worker bloque	Job en cours trop long	Timeout configurable, retry
JSON segments invalide	JSONField corrupt	Validation Django ORM
Export echoue	Donnees manquantes	Verification avant generation

Toutes les erreurs sont logguees dans les logs Django (console et fichier). Les erreurs 500 sont affichees dans le template d'erreur Django en mode debug.